



# UNINASSAU

## ÁRVORES, TÉCNICAS DE PESQUISA E TÉCNICAS DE ORDENAÇÃO

Análise e Desenvolvimento de Sistemas  
Programação Orientada à Objetos e Estruturas de dados  
Profº Eng. Esp. Lindenberg Andrade

# ÁRVORES

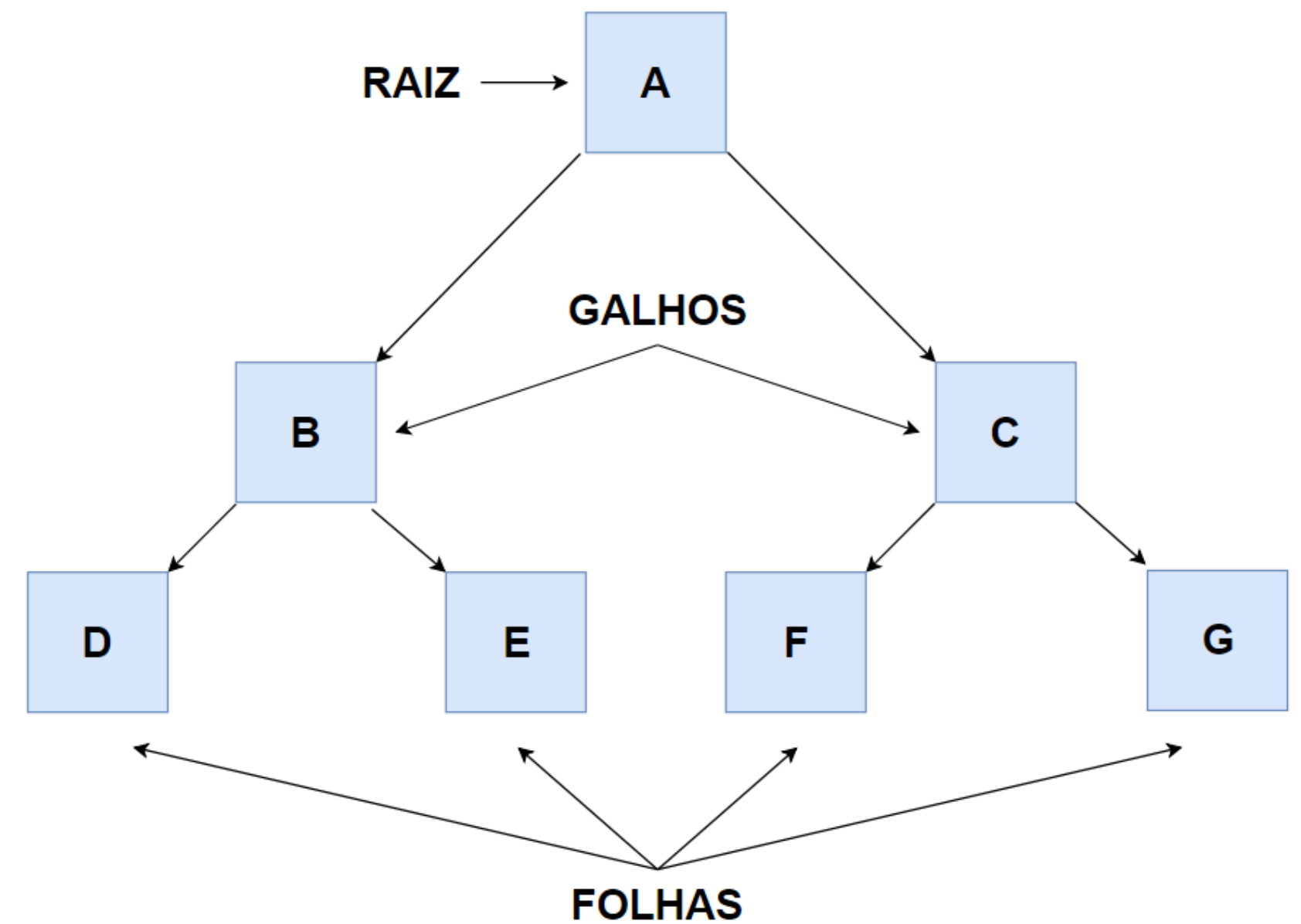
# O QUE É UMA ÁRVORE?

Uma árvore é uma estrutura hierárquica composta por nós conectados por arestas.

O primeiro nó é a raiz; cada nó pode ter filhos; nós sem filhos são folhas.

- **Características**

- Estrutura não linear
- Representa relações hierárquicas
- Permite operações eficientes de busca



Estrutura hierárquica  
Composta por nós  
Raiz, filhos, folhas

Aplicações: compilers, jogos, bancos de dados

# ÁRVORE BINÁRIA

Cada nó possui no máximo dois filhos: esquerdo e direito.

Cada nó possui até dois filhos

Tipos:

- Árvore binária comum
- Árvore binária de Pesquisa (BST)

```
class No {  
    int valor;  
    No esquerdo, direito;  
  
    No(int v) {  
        valor = v;  
        esquerdo = direito = null;  
    }  
}
```



# ÁRVORE BINÁRIA

## Inserção na Árvore Binária de Pesquisa (BST)

- **A BST garante:**
  - Valores menores → vão para a esquerda
  - Valores maiores → vão para a direita

```
class ArvoreBST {  
    No raiz;  
  
    void inserir(int v) {  
        raiz = inserirRec(raiz, v);  
    }  
  
    No inserirRec(No raiz, int v) {  
        if (raiz == null) return new No(v);  
  
        if (v < raiz.valor)  
            raiz.esquerdo = inserirRec(raiz.esquerdo, v);  
        else if (v > raiz.valor)  
            raiz.direito = inserirRec(raiz.direito, v);  
  
        return raiz;  
    }  
}
```

# PERCURSOS EM ÁRVORES

## Pré-Ordem (Root → Left → Right)

```
void preOrdem(No n) {  
    if (n != null) {  
        System.out.print(n.valor + " ");  
        preOrdem(n.esquerdo);  
        preOrdem(n.direito);  
    }  
}
```

# PERCURSOS EM ÁRVORES

## Em-Ordem (Left → Root → Right)

Retorna valores ordenados em uma BST

```
void emOrdem(No n) {  
    if (n != null) {  
        emOrdem(n.esquerdo);  
        System.out.print(n.valor + " ");  
        emOrdem(n.direito);  
    }  
}
```

## Pós-Ordem (Left → Right → Root)

```
void posOrdem(No n) {  
    if (n != null) {  
        posOrdem(n.esquerdo);  
        posOrdem(n.direito);  
        System.out.print(n.valor + " ");  
    }  
}
```

# TÉCNICAS DE PESQUISA

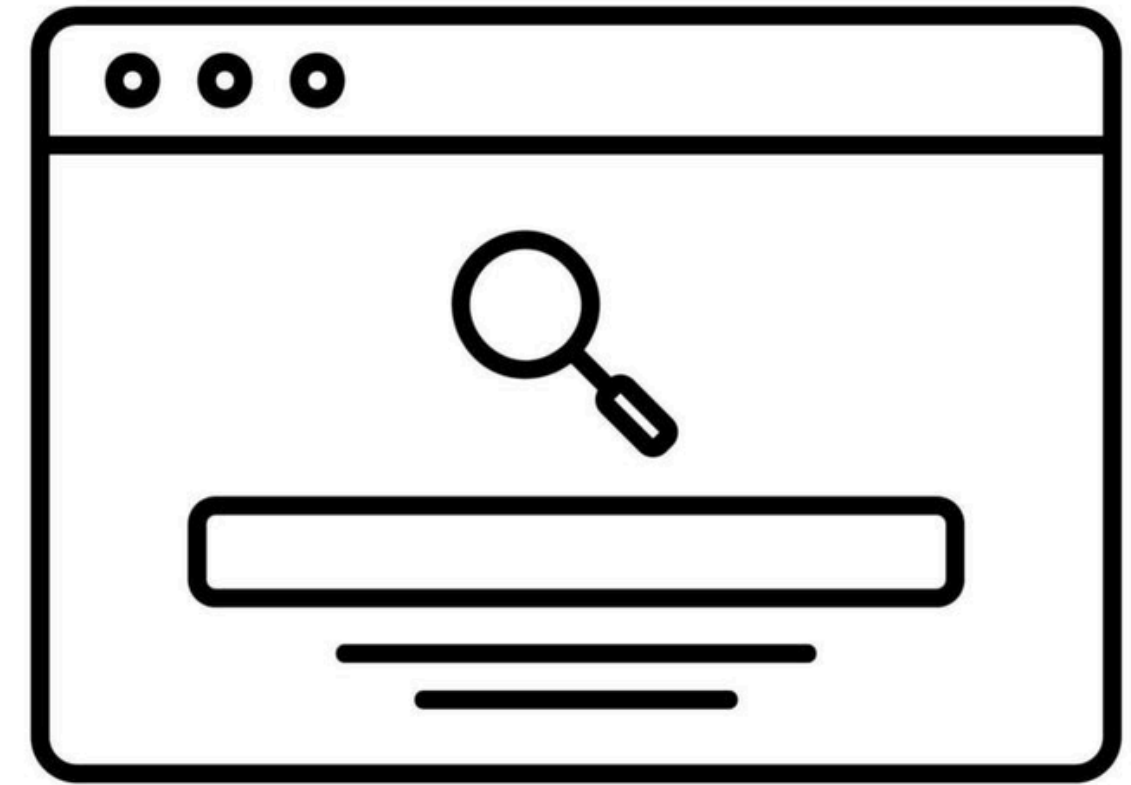


# O QUE SÃO TÉCNICAS DE PESQUISA?

Técnicas de pesquisa são métodos usados para procurar um elemento dentro de uma coleção de dados, como vetores, listas, árvores, arquivos, bancos de dados etc.

Elas servem para responder perguntas como:

- “O elemento X está presente?”
- “Em qual posição ele está?”
- “Existe algum valor maior/menor que Y?”





# PESQUISA LINEAR (LINEAR SEARCH)

É a técnica mais simples:

Percorre elemento por elemento até encontrar o valor buscado.

- **Características**

- Funciona em **qualquer estrutura**
- Não precisa estar ordenada
- **Lenta** para grandes volumes

[5, 3, 9, 1, 4]

Para procurar o 1, ela examina: 5  
→ 3 → 9 → 1 → encontrou.

```
int buscaLinear(int[] v, int x) {  
    for (int i = 0; i < v.length; i++) {  
        if (v[i] == x) return i;  
    }  
    return -1;  
}
```

# PESQUISA LINEAR (BINARY SEARCH)

É muito mais rápida, porém **exige dados ordenados**.

Funciona dividindo o problema pela metade repetidamente.

- **Características**

- Requer **vetor ordenado**
- Muito rápida

Como funciona:

Se você procura o número 30 em:

[10, 20, 30, 40, 50]

1. Olha o meio → 30

2. Encontrou!

Ou se fosse 25:

1. Meio = 30 → 25 é menor → busca na metade esquerda

2. Reduz o problema

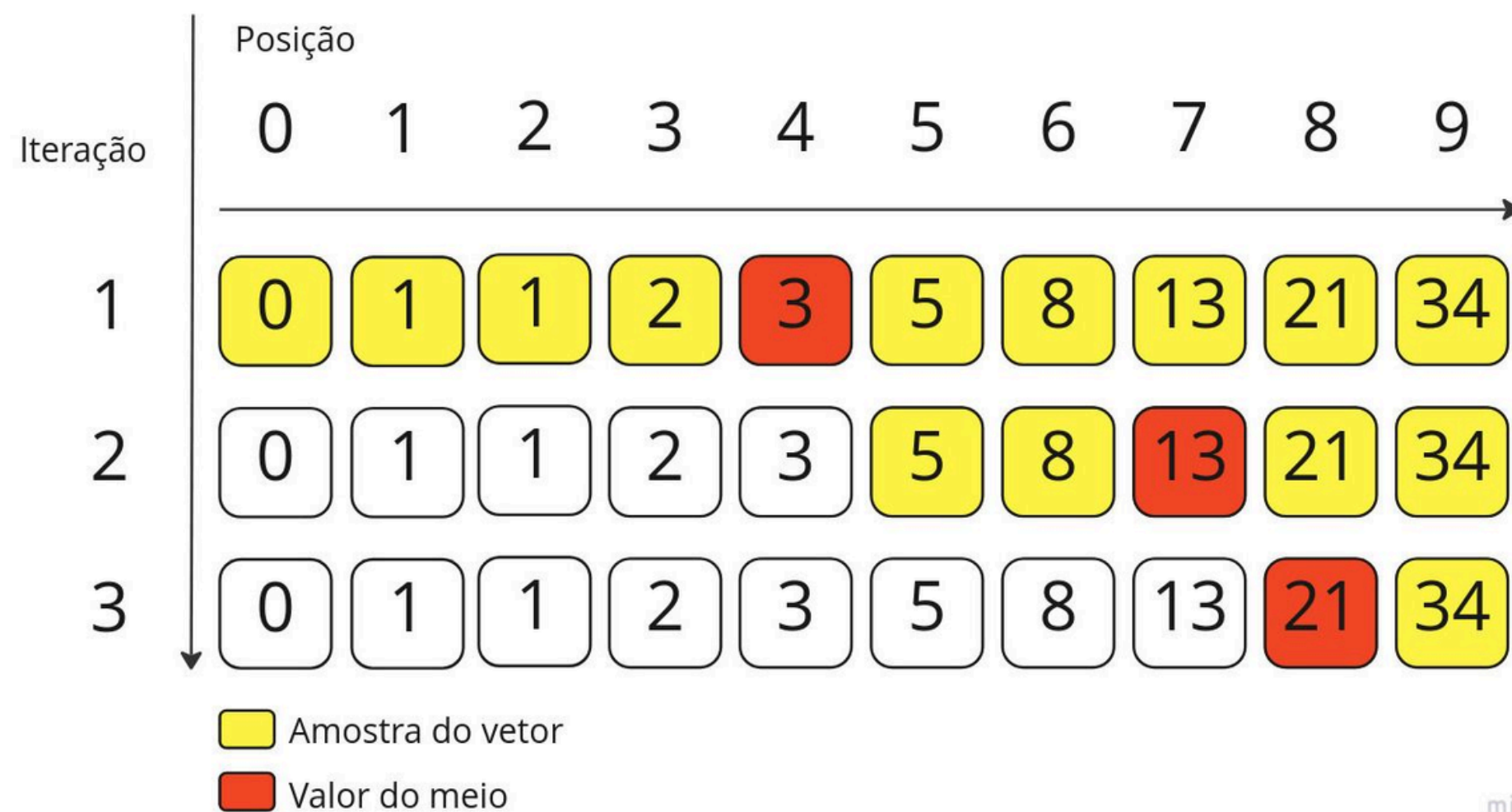
3. Continua até achar ou terminar

```
int buscaLinear(int[] v, int x) {  
    for (int i = 0; i < v.length; i++) {  
        if (v[i] == x) return i;  
    }  
    return -1;  
}
```

# PESQUISA BINÁRIA

Requer **array ordenado**.

Divide o problema pela metade repetidamente.



```
int buscaBinaria(int[] v, int x) {  
    int inicio = 0, fim = v.length - 1;  
  
    while (inicio <= fim) {  
        int meio = (inicio + fim) / 2;  
  
        if (v[meio] == x) return meio;  
        else if (x < v[meio])  
            fim = meio - 1;  
        else  
            inicio = meio + 1;  
    }  
    return -1;  
}
```

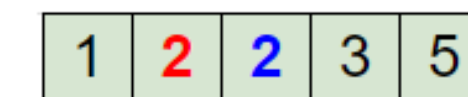
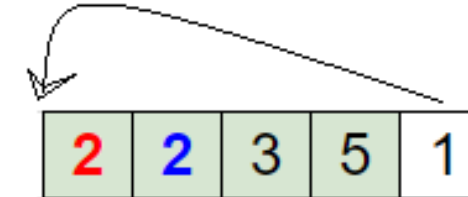
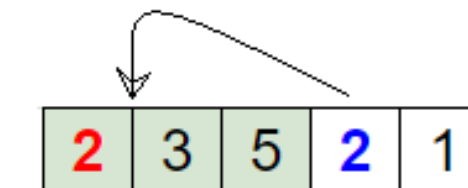
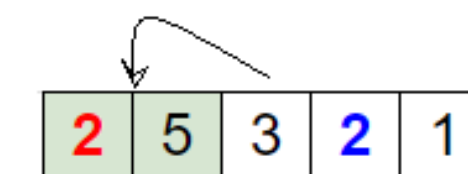
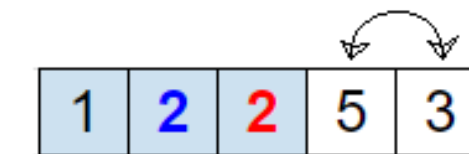
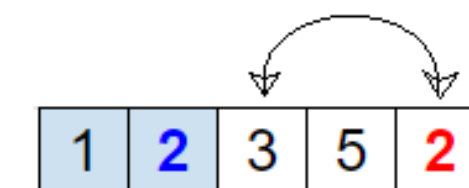
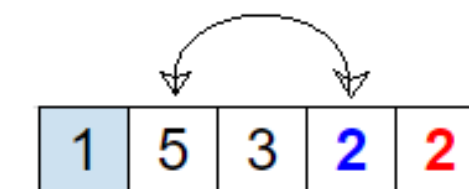
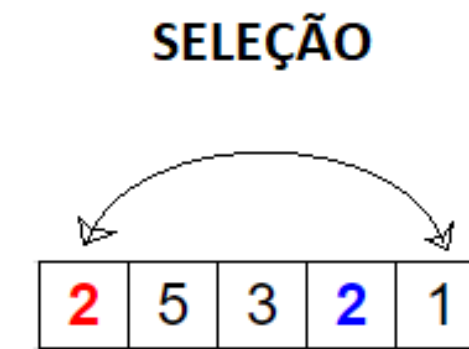
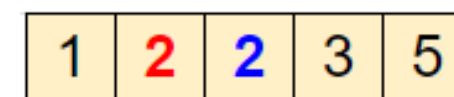
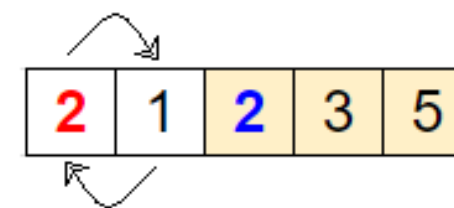
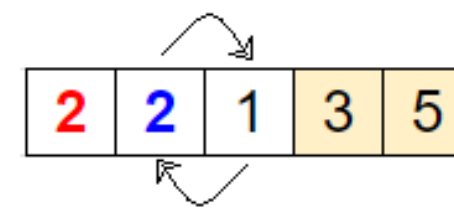
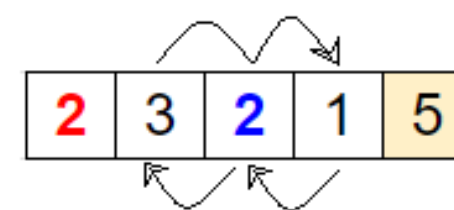
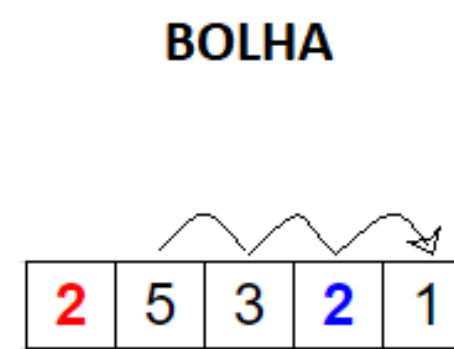
# TÉCNICAS DE ORDENAÇÃO



# O QUE SÃO TÉCNICAS DE ORDENAÇÃO?

Técnicas de ordenação (ou algoritmos de ordenação) são métodos usados para reorganizar elementos de uma lista ou vetor em uma ordem específica, geralmente crescente ou decrescente.

Ordenar dados é essencial para buscas rápidas, organização de relatórios, otimização de algoritmos etc.

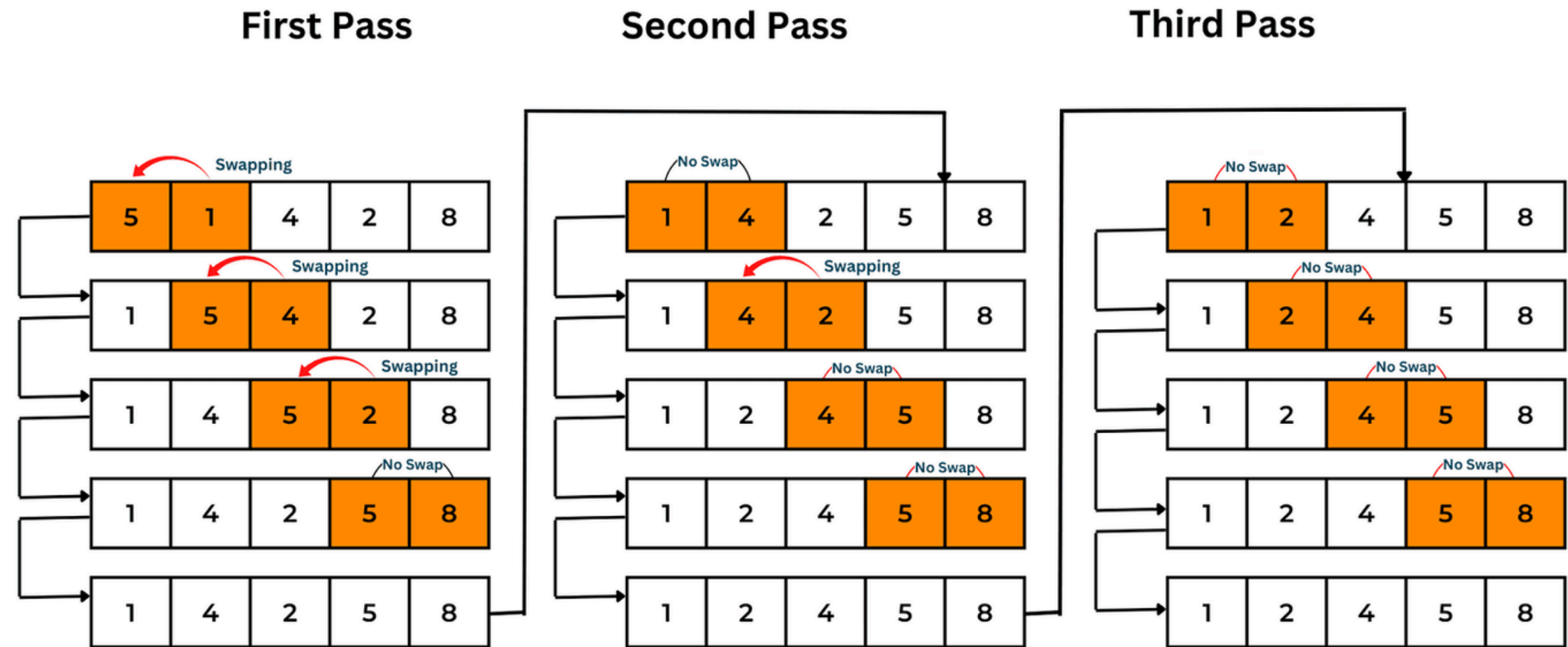


# BUBBLE SORT

Compara elementos vizinhos e vai “borbulhando” os maiores para o final.

- **Vantagem:** simples
- **Desvantagem:** lento para dados grandes

## BUBBLE SORTING



# BUBBLE SORT

Compara elementos vizinhos e vai “borbulhando” os maiores para o final.

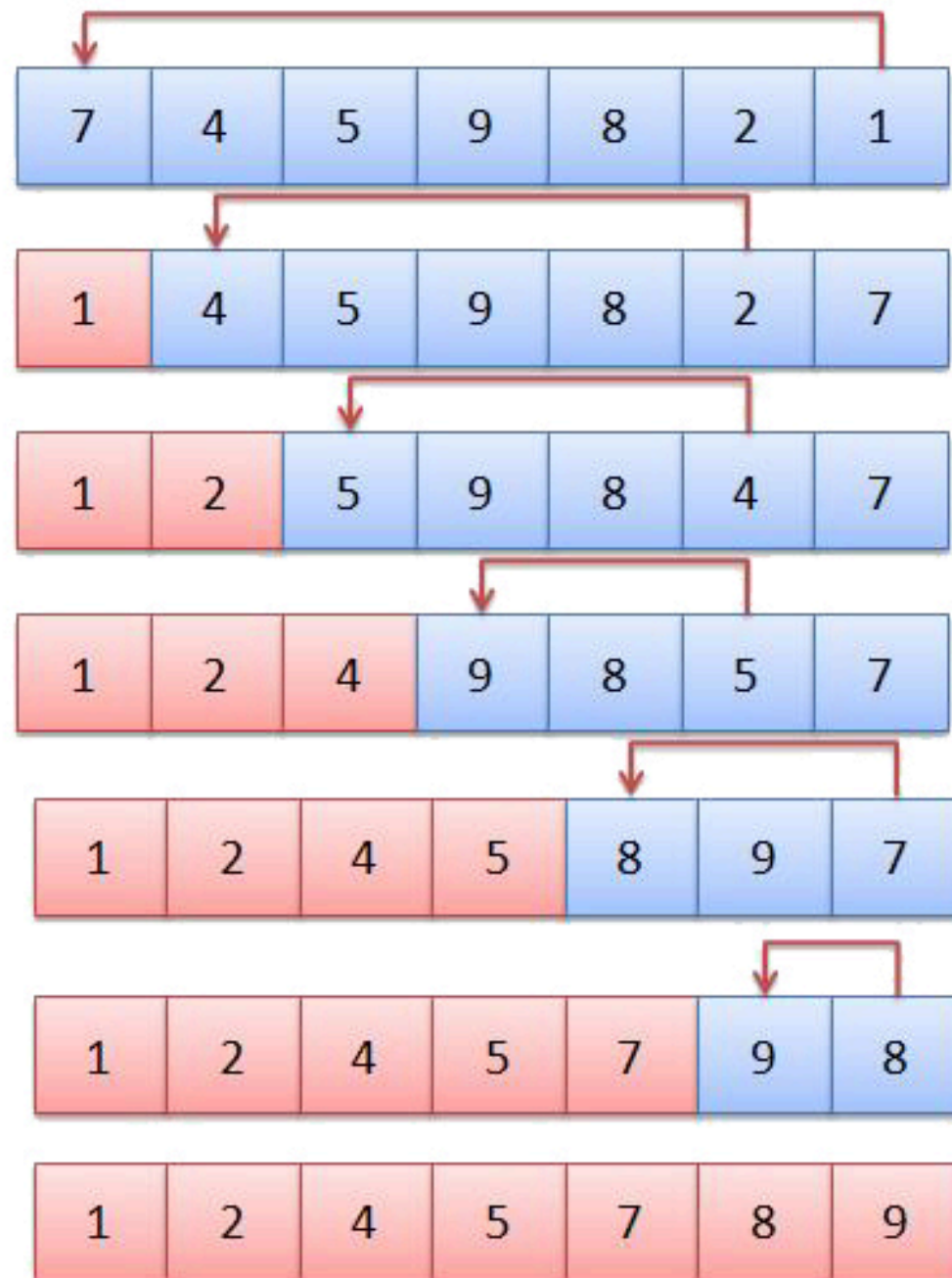
- **Vantagem:** simples
- **Desvantagem:** lento para dados grandes
- **Complexidade:**  $O(n^2)$

```
void bubbleSort(int[] v) {  
    for (int i = 0; i < v.length - 1; i++) {  
        for (int j = 0; j < v.length - 1 - i; j++) {  
            if (v[j] > v[j + 1]) {  
                int temp = v[j];  
                v[j] = v[j + 1];  
                v[j + 1] = temp;  
            }  
        }  
    }  
}
```



# SELECTION SORT

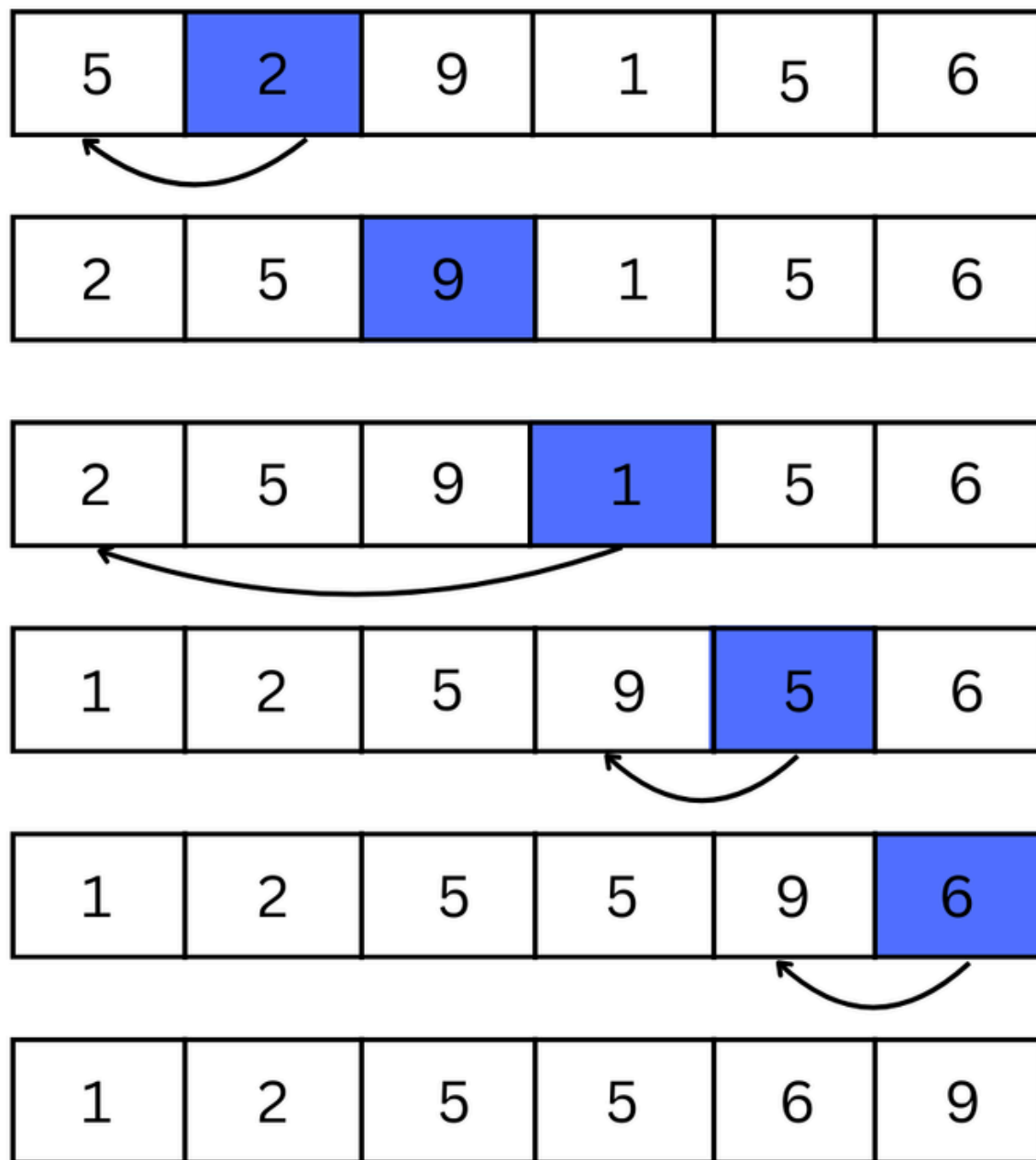
Selecione o menor elemento e coloque na posição correta.



```
void selectionSort(int[] v) {  
    for (int i = 0; i < v.length - 1; i++) {  
        int min = i;  
        for (int j = i + 1; j < v.length; j++) {  
            if (v[j] < v[min])  
                min = j;  
        }  
        int temp = v[i];  
        v[i] = v[min];  
        v[min] = temp;  
    }  
}
```

# INSERTION SORT

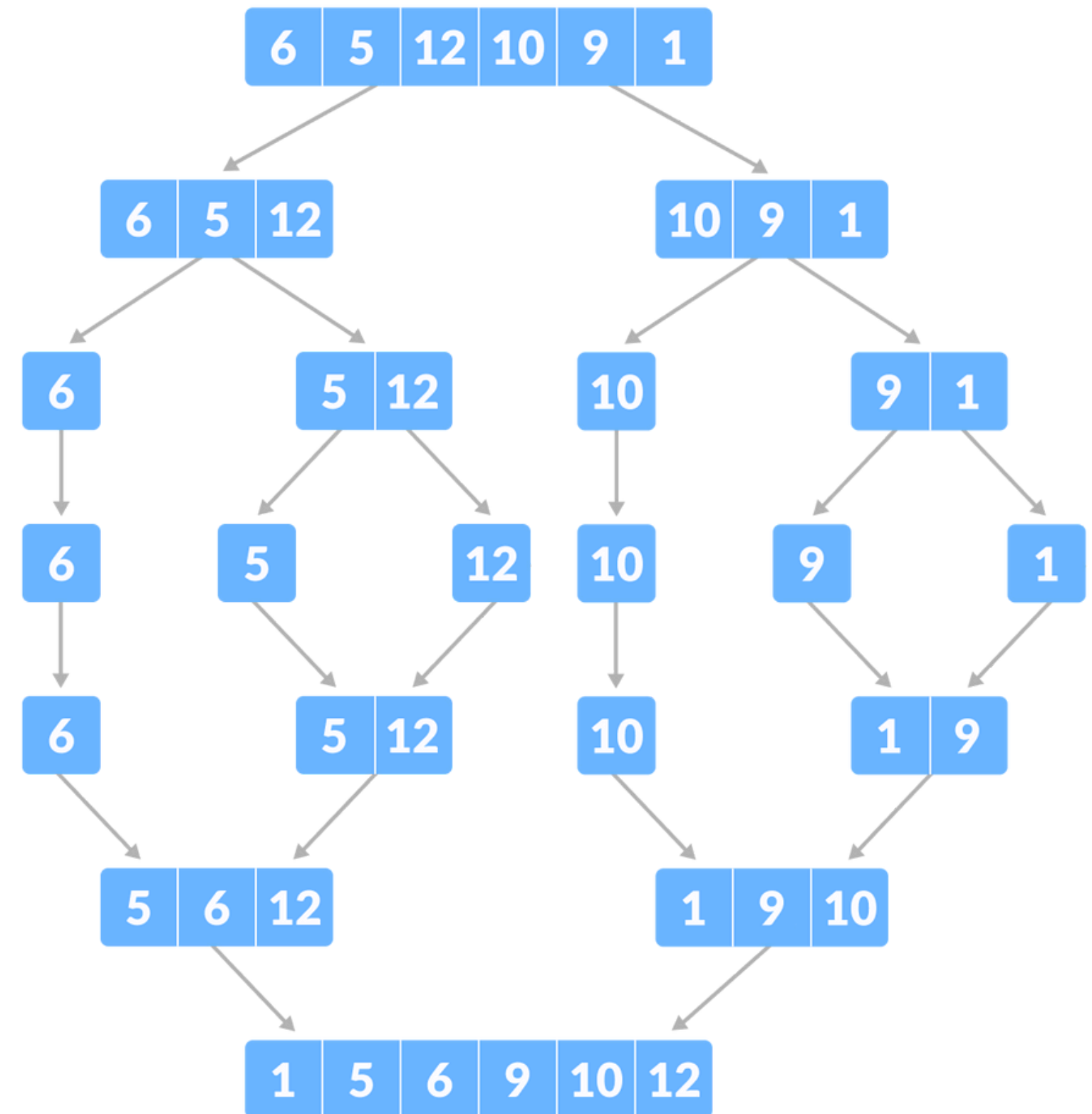
Insere cada elemento na posição correta.



```
void insertionSort(int[] v) {  
    for (int i = 1; i < v.length; i++) {  
        int chave = v[i];  
        int j = i - 1;  
  
        while (j >= 0 && v[j] > chave) {  
            v[j + 1] = v[j];  
            j--;  
        }  
        v[j + 1] = chave;  
    }  
}
```

# MERGE SORT (DIVIDIR E CONQUISTAR)

Usa dividir-para-conquistar: divide, ordena as partes e junta.



# MERGE SORT (DIVIDIR E CONQUISTAR)

Mais eficiente e recursivo

```
void mergeSort(int[] v, int ini, int fim) {  
    if (ini < fim) {  
        int meio = (ini + fim) / 2;  
  
        mergeSort(v, ini, meio);  
        mergeSort(v, meio + 1, fim);  
        merge(v, ini, meio, fim);  
    }  
}
```

```
void merge(int[] v, int ini, int meio, int fim) {  
    int[] temp = new int[fim - ini + 1];  
    int i = ini, j = meio + 1, k = 0;  
  
    while (i <= meio && j <= fim) {  
        if (v[i] <= v[j]) temp[k++] = v[i++];  
        else temp[k++] = v[j++];  
    }  
  
    while (i <= meio) temp[k++] = v[i++];  
    while (j <= fim) temp[k++] = v[j++];  
  
    for (i = ini, k = 0; i <= fim; i++, k++)  
        v[i] = temp[k];  
}
```

# EXEMPLOS



# EXEMPLO 1: EXEMPLO DE ÁRVORE (BST)

## Problema:

Inserir os valores 8, 3, 10, 1, 6 em uma Árvore Binária de Busca e exibir o percurso em-ordem (que deve produzir valores ordenados).

```
class No {  
    int valor;  
    No esquerdo, direito;  
  
    No(int valor) {  
        this.valor = valor;  
        esquerdo = direito = null;  
    }  
}
```

```
class ArvoreBST {  
    No raiz;  
  
    // Inserção recursiva  
    No inserir(No raiz, int valor) {  
        if (raiz == null) return new No(valor);  
        if (valor < raiz.valor)  
            raiz.esquerdo = inserir(raiz.esquerdo, valor);  
        else  
            raiz.direito = inserir(raiz.direito, valor);  
        return raiz;  
    }  
  
    // Em-ordem  
    void emOrdem(No raiz) {  
        if (raiz != null) {  
            emOrdem(raiz.esquerdo);  
            System.out.print(raiz.valor + " ");  
            emOrdem(raiz.direito);  
        }  
    }  
}
```



# EXEMPLO 1: EXEMPLO DE ÁRVORE (BST)

```
public class ExemploArvore {  
    public static void main(String[] args) {  
  
        ArvoreBST arvore = new ArvoreBST();  
  
        int[] valores = {8, 3, 10, 1, 6};  
  
        for (int v : valores)  
            arvore.raiz = arvore.inserir(arvore.raiz, v);  
  
        System.out.print("Percurso em-ordem: ");  
        arvore.emOrdem(arvore.raiz);  
    }  
}
```

Saída esperada:

1 3 6 8 10

# EXEMPLO 2: EXEMPLO DE TÉCNICA DE PESQUISA

## Problema:

Buscar o número 7 no vetor {2, 4, 7, 9, 13} usando Pesquisa Binária.

```
public class PesquisaBinaria {  
  
    public static int buscaBinaria(int[] v, int x) {  
        int inicio = 0, fim = v.length - 1;  
  
        while (inicio <= fim) {  
            int meio = (inicio + fim) / 2;  
  
            if (v[meio] == x) return meio;  
            if (x < v[meio])  
                fim = meio - 1;  
            else  
                inicio = meio + 1;  
        }  
        return -1;  
    }  
}
```

```
public static void main(String[] args) {  
    int[] vetor = {2, 4, 7, 9, 13};  
    int elemento = 7;  
  
    int pos = buscaBinaria(vetor, elemento);  
  
    if (pos != -1)  
        System.out.println("Elemento encontrado na posição: " + pos);  
    else  
        System.out.println("Elemento não encontrado");  
}  
}
```

Saída esperada:

Elemento encontrado na posição: 2

# EXEMPLO 3: EXEMPLO DE TÉCNICA DE ORDENAÇÃO

## Problema:

Ordenar o vetor {5, 1, 4, 2, 8} usando **Insertion Sort**.

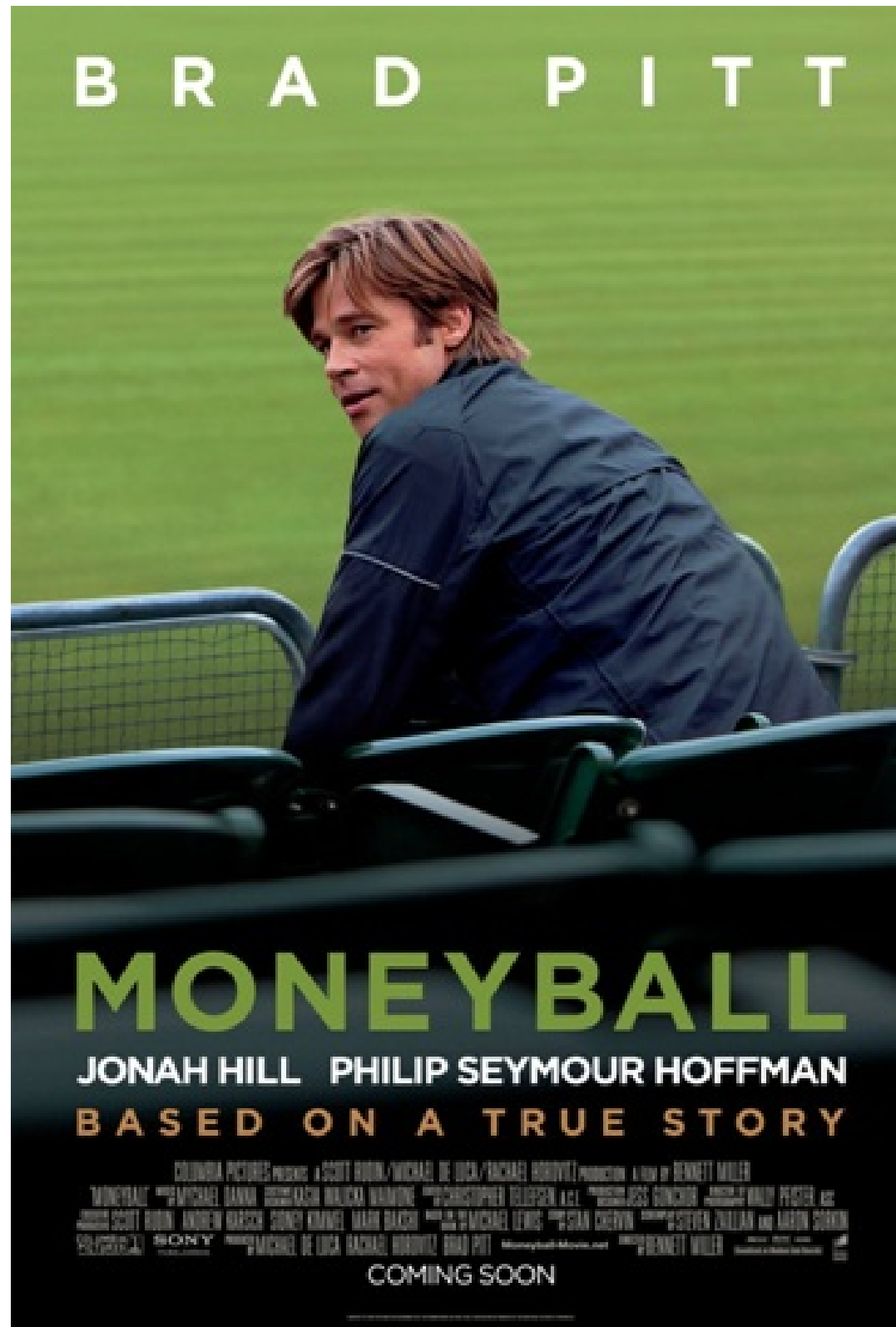
```
public class InsertionSortExemplo {  
  
    public static void insertionSort(int[] v) {  
        for (int i = 1; i < v.length; i++) {  
            int chave = v[i];  
            int j = i - 1;  
  
            while (j >= 0 && v[j] > chave) {  
                v[j + 1] = v[j];  
                j--;  
            }  
            v[j + 1] = chave;  
        }  
    }  
}
```

```
public static void main(String[] args) {  
    int[] vetor = {5, 1, 4, 2, 8};  
  
    insertionSort(vetor);  
  
    System.out.print("Vetor ordenado: ");  
    for (int n : vetor)  
        System.out.print(n + " ");  
}
```

Saída esperada:

1 2 4 5 8

# O homem que mudou o jogo (Moneyball) 2011



O filme de Bennett Miller é baseado na história de Billy Beane (Brad Pitt), gerente geral do time de beisebol do Oakland Athletics. História que, inclusive, foi descrita no livro *The Art of Winning an Unfair Game* de Michael Lewis. A história se passa no ano de 2002, período em que a equipe do Oakland iniciaria a temporada com uma situação financeira extremamente desfavorável. Beane sabia que poderia driblar esse cenário, utilizando uma poderosa ferramenta: os dados!

Por meio de uma profunda análise estatística dos jogadores, Beane determinou novas estratégias que pudessem ser colocadas em prática durante os jogos.

Tendo em mãos dados que traziam os previsíveis desempenhos dos atletas, tornou-se possível organizar a equipe para que pudessem jogar de forma estratégica, com base na falha dos adversários.

# Dúvidas?



Profº Lindenberg Andrade  
E-mail: linndemberg1@gmail.com



Additional contacts via QR code